STUDENT EXPENSE TRACKER

VITyarthi - Build Your Own Project

Project Report

Prepared as a beginner-level Python project

Technology: Python 3 | Storage: JSON | Interface: Terminal

1. Introduction

The Student Expense Tracker is a small terminal-based Python application. The main idea is to help a college student write down daily expenses and understand how much money has been spent. The application is kept simple so that the main Python concepts can be understood by a first-year student.

2. Problem Statement

Students have many small expenses such as food, transport, books, fees and shopping. If these expenses are not recorded, it becomes difficult to calculate the total spending or check whether a monthly budget is being followed. This project provides a simple local solution for recording and checking expenses.

3. Objectives

• Create a simple system for recording daily expenses.

• Allow the user to view and delete saved expenses.

• Provide monthly and category-wise reports.

• Allow the user to set a monthly budget.

• Store information in a local JSON file.

• Practice Python functions, classes, lists, dictionaries and file handling.

4. Functional Requirements

Module	What it does
Expense Entry	Takes date, amount, category and note.
Expense Viewing	Displays all saved expenses and the total.
Reports	Shows monthly and category-wise spending.
Expense Deletion	Deletes an expense using its ID.
Budget	Stores a monthly budget and calculates remaining money.
Storage	Loads and saves data using JSON.

5. Non-Functional Requirements

• Usability: The menu and questions should be easy for a beginner to follow.

• Reliability: Wrong input should show a message instead of stopping the program.

• Maintainability: The program is split into small files with clear purposes.

• Resource Efficiency: The project uses the Python standard library and a small local JSON file.

• Privacy: Data stays in the local project folder and is not sent to an online service.

6. System Architecture

The project follows a simple layered structure. The main file handles the menu. The manager handles expense operations. The storage file handles JSON data. Validation checks user input and the reports file prepares simple reports.

User	Menu (main.py)	Expense Manager
	↓	↓
	Validators	JSON Storage
	↓	↓
	Reports	expenses.json

7. Workflow Diagram

Start
↓
Show Menu
↓
User Selects Option
↓
Validate Input
↓
Perform Operation
↓
Save / Display Result
↓
Return to Menu or Exit

8. Use Case Diagram

The main actor is the student using the terminal program.

Student	→	Add Expense
Student	→	View Expenses
Student	→	Monthly Report
Student	→	Category Report
Student	→	Delete Expense
Student	→	Set Budget
Student	→	View Budget Status

9. Class / Component Diagram

Expense	ExpenseManager	Storage
id	add_expense()	load_data()
date	get_all_expenses()	save_data()
amount	delete_expense()	
category	set_budget()	
note	total_expenses()	

10. Sequence Diagram

Student	main.py	ExpenseManager	storage.py
Select Add	asks for data		
Enter data	calls add_expense()	creates Expense	saves JSON
	shows success		

11. Storage / ER-style Design

The project uses JSON instead of a database. Each expense contains an ID, date, amount, category and note. The JSON file also contains the monthly budget.

expenses	Field	Type
	id	Integer
	date	String
	amount	Number
	category	String
	note	String
budget	monthly budget	Number

12. Design Decisions and Rationale

• Python was selected because it is simple and suitable for learning programming basics.

• JSON was selected because it is easy to read and does not require a database server.

• The project uses separate Python files so each part has one main purpose.

• A terminal menu was selected because it keeps the interface simple and focuses on programming concepts.

• Basic validation was added so common input mistakes do not immediately crash the program.

13. Implementation Details

The program starts from main.py. The ExpenseManager class loads existing data and provides methods for adding, viewing, deleting and calculating expenses. The validators.py file checks dates, amounts, categories and notes. The reports.py file contains report functions. The storage.py file reads and writes expenses.json.

14. Testing Approach

Test	Expected result
Valid expense	Expense is saved.
Wrong amount	Program asks for a valid number.
Negative amount	Program rejects the amount.
Wrong date	Program asks for YYYY-MM-DD.
Wrong menu choice	Program shows an invalid choice message.
Delete unknown ID	Program says the ID was not found.
Load saved file	Previously saved data is available.

A small unittest file is also included in the project folder.

15. Screenshots / Results

The supplied project is a terminal application, so the report includes sample terminal results rather than fabricated screenshots. After running the program in VS Code, the student can add screenshots of the menu, expense list and reports to the report if required.

ID	Date	Category	Amount	Note
1	2026-09-28	Food	Rs. 120.00	Canteen lunch
2	2026-09-29	Transport	Rs. 80.00	Bus

16. Challenges Faced

• Keeping the project simple while still meeting the required modules.

• Handling incorrect dates and amounts entered by the user.

• Making sure saved JSON data can be loaded again.

• Dividing the program into separate files without making the structure difficult to understand.

17. Learnings and Key Takeaways

• I learned how to divide a program into multiple Python files.

• I learned how classes can be used to manage related data and operations.

• I learned basic file handling using JSON.

• I learned why input validation is important in a real program.

• I learned how a simple requirement can be converted into a working application.

18. Future Enhancements

• Edit an existing expense.

• Export expenses to CSV.

• Add graphs for spending.

• Add weekly and yearly reports.

• Add a simple graphical interface.

• Add user login if the project later needs multiple users.

19. References

1. VITyarthi - Build Your Own Project: General Project Instructions & Submission Guidelines (uploaded course PDF).

2. Python standard library documentation concepts used by the project: json, pathlib, datetime and unittest.

3. The supplied Student Expense Tracker project files were used as the starting reference for the project idea and features.